

OGX: An Open-Source, Vendor-Neutral Generative AI Application Server

Francisco Javier Arceo¹✉, Sébastien Han¹, Matthew Farrellee³, Charlie Doern¹, Yuan Tang¹, Derek Higgins¹, Varsha Prasad Narsing¹, Gordon Sim¹, Sumanth Kamenani¹, Ben Browning¹, and Raghotham Murthy²

¹ Red Hat AI, USA ² Meta, USA ³ Independent ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: [Fangzhou Xie](#) ✉

Reviewers:

- [@showdawn](#)
- [@alexzerg](#)

Submitted: 05 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

OGX (Open GenAI Stack), formerly Llama Stack ([Llama Stack Contributors, 2025](#)), is an open-source AI application server and Python library that implements the APIs of major frontier labs (OpenAI, Anthropic, Google) with pluggable backend providers ([OGX Contributors, 2026b](#)). Teams building agentic AI applications—such as retrieval-augmented generation (RAG) pipelines, multi-turn conversational agents, and tool-calling workflows—can develop against a single, stable API surface and deploy with any combination of inference engine, vector database, and safety backend, without changing application code.

OGX's primary API focus is the Responses API for server-side agentic orchestration, conforming to the Open Responses specification ([Open Responses Community, 2026](#)). The server also exposes Chat Completions, Embeddings, Vector Stores, Files, and Batches endpoints. Beyond OpenAI compatibility, OGX natively supports the Anthropic Messages API (/v1/messages) and Google GenAI Interactions API (/v1alpha/interactions), allowing teams using any of the three major client SDKs to connect to the same server. OGX supports over 20 inference providers (including vLLM, Ollama, OpenAI, Anthropic, Bedrock, and Gemini), 13 vector store backends, and 7 safety providers. It can run as an HTTP server for production deployments or be imported directly as a Python library for scripting and notebooks. A companion Kubernetes Operator ([OGX Contributors, 2026a](#)) automates deployment lifecycle management through custom resources, supporting multi-architecture builds, hot-swappable distribution images, and both shared and per-tenant isolation topologies. Together, OGX and its operator serve as the self-hosted, model-agnostic backend for AI-powered developer tools such as Claude Code, Codex CLI, OpenCode, and OpenHands.

Statement of Need

AI application development today is tightly coupled to proprietary API providers. While inference-only workloads can increasingly be swapped across providers—vLLM, for example, supports the Responses API for basic inference—applications that rely on the full stack (retrieval, tool calling, conversation state, safety guardrails) remain difficult to migrate without rewriting significant application logic. This coupling limits reproducibility, makes comparisons across model providers difficult, and prevents teams from running AI workloads on controlled infrastructure—a requirement in regulated, privacy-sensitive, and air-gapped environments.

Existing open-source tools address parts of this problem but not the whole. Inference engines like vLLM ([Kwon et al., 2023](#)) and SGLang ([Zheng et al., 2024](#)) serve models efficiently but do not provide retrieval, tool calling, or conversation management. Gateway proxies like LiteLLM route requests across providers but do not execute the agentic loop or manage vector stores. Client-side frameworks like LangChain ([LangChain, Inc., 2023](#)) and LangGraph ([LangChain,](#)

42 [Inc., 2024](#)) provide developer abstractions but push orchestration, state management, and
43 security enforcement to the application layer.

44 OGX fills this gap by providing a complete, self-hosted AI application server that implements
45 the OpenAI API surface—with the Responses API as its primary focus—alongside Anthropic
46 Messages and Google GenAI Interactions compatibility layers. Developers write code against
47 standard endpoints (`/v1/responses`, `/v1/chat/completions`, `/v1/vector_stores`) and swap
48 the underlying infrastructure through configuration, not code changes. This decouples three
49 decisions that are currently entangled: which SDK to use, which model to run, and where to
50 deploy.

51 State of the Field

52 The AI application ecosystem has stratified into layers that each solve a subset of the deployment
53 problem. OGX continues the Llama Stack project under a renamed, model-agnostic mission;
54 the relevant comparison is therefore the server-side API layer that OGX provides versus adjacent
55 inference engines, gateways, and client-side frameworks.

56 **Inference engines** (vLLM ([Kwon et al., 2023](#)), SGLang ([Zheng et al., 2024](#)), Ollama) focus on
57 efficient model serving. They optimize throughput and latency but do not provide retrieval,
58 conversation state, tool execution, or safety guardrails. An application using vLLM for inference
59 must separately integrate a vector database, implement its own agentic loop, and manage
60 multi-turn state.

61 **API gateways** (LiteLLM, OpenRouter) provide a unified interface across multiple inference
62 providers but act as pass-through proxies. They do not manage vector stores, execute tool
63 calls, or maintain conversation history—they translate request formats between SDKs and
64 providers.

65 **Client-side frameworks** (LangChain ([LangChain, Inc., 2023](#)), LangGraph ([LangChain, Inc., 2024](#)),
66 LlamaIndex ([LlamaIndex, 2022](#)), CrewAI ([CrewAI, Inc., 2024](#)), Haystack ([deepset, 2023](#)))
67 provide rich developer abstractions for building agents and RAG pipelines. However,
68 they execute orchestration client-side, distributing security-critical logic across application code.
69 These frameworks are complementary to OGX: they compose agent logic while OGX provides
70 the server-side execution target they call into.

71 **Proprietary platforms** (OpenAI's Responses API ([OpenAI, 2025](#)), Databricks Mosaic AI
72 ([Databricks, 2025](#))) offer integrated experiences but couple applications to a specific vendor's
73 infrastructure and pricing.

74 OGX occupies a distinct position: a self-hosted, vendor-neutral server that implements the full
75 API surface—inference, retrieval, tool execution, conversation management, and safety—with
76 pluggable providers at every layer. Its conformance to the Open Responses specification ([Open
77 Responses Community, 2026](#)) ensures interoperability with any client that speaks the same
78 protocol. It does not compete with client-side frameworks; it is the infrastructure they deploy
79 against when reproducibility, provider portability, and centralized policy enforcement matter.

80 Software Design

81 Provider Architecture

82 OGX's core abstraction is the pluggable provider. Each API capability (inference, vector storage,
83 safety, tool runtime, file processing) is defined by a Protocol interface in the lightweight `ogx-
84 api` package, allowing third-party providers to implement the contract without depending
85 on the full server. Concrete providers implement these interfaces for specific backends:
86 `remote::openai` and `remote::anthropic` for hosted APIs, `remote::vllm` for self-hosted GPU
87 inference, `inline::faiss` and `remote::pgvector` for vector search, and so on. A routing layer

88 dispatches requests to provider instances based on logical resource identifiers, enabling multiple
89 providers to serve the same API simultaneously—for example, Ollama handling local models
90 while OpenAI handles hosted models, both behind `/v1/chat/completions`.

91 A *distribution* packages a specific set of providers and configuration into a deployable unit,
92 decoupling application logic from infrastructure selection. This design favors reproducible
93 configuration over ad hoc application code: teams can prototype with lightweight inline
94 providers (Ollama, `sqlite-vec` (Garcia, 2024)) and deploy to production backends (vLLM,
95 `pgvector`) by changing the distribution, not the application.

96 Dual Deployment Model

97 OGX runs in two modes. **Server mode** exposes HTTP endpoints accessible from any language
98 or tool. **Library mode** allows direct Python import with zero network overhead, suitable for
99 notebooks and scripts. Both modes use identical provider routing and API semantics.

100 Multi-SDK Compatibility

101 OGX serves three client SDK protocols from a single server. The **OpenAI-compatible endpoints**
102 (`/v1/chat/completions`, `/v1/responses`, `/v1/vector_stores`) are the primary interface and
103 the Responses API implementation conforms to the Open Responses specification (Open
104 Responses Community, 2026). The **Anthropic Messages endpoint** (`/v1/messages`) and
105 **Google GenAI Interactions endpoint** (`/v1alpha/interactions`) provide native compatibility
106 for teams using those SDKs. This portability comes with a trade-off: OGX must normalize
107 provider-specific behavior into stable API contracts while still exposing enough backend-specific
108 configuration for real deployments.

109 Server-Side Agentic Orchestration

110 The Responses API implements server-side agentic orchestration: the inference-tool-inference
111 loop executes within the server process, not the client (OpenAI, 2025). This centralizes security
112 enforcement, tool authorization, and conversation state management, at the cost of moving
113 some flexibility from application code into server configuration. Built-in tools include file search
114 (RAG over vector stores), web search, code interpretation, and Model Context Protocol (MCP)
115 (Anthropic, 2026) integration for external tool servers.

116 OGX is extending server-side execution with a Containers API and a Skills API. The Containers
117 API (`/v1/containers`) manages sandboxed execution environments—matching the OpenAI
118 Containers API—enabling models to execute shell commands in isolated Docker, Podman, or
119 Kubernetes containers via a `shell` tool in the Responses API. The Skills API (`/v1alpha/skills`)
120 manages versioned skill bundles that package tools, prompts, and configuration into reusable,
121 composable units. Together, these APIs move code execution and task composition onto the
122 server, extending the same trust-boundary principle that governs retrieval and tool authorization.

123 Unlike inference engines and API gateways that treat requests as stateless, OGX manages
124 state natively: the Conversations API persists multi-turn history with tenant-scoped isolation,
125 the Prompts API provides versioned prompt templates, and a resource registry tracks models,
126 vector stores, and files as first-class server objects. A Compaction API summarizes long histories
127 to manage context window limits. Telemetry is built on OpenTelemetry (OTEL) with MLflow
128 (Zaharia et al., 2018) tracing integration for logging spans, tool calls, and retrieval steps to
129 existing ML experiment tracking infrastructure. This state and observability layer is what
130 makes OGX a complete application server rather than a stateless proxy.

131 For multitenant deployments, OGX provides attribute-based access control (ABAC) that
132 enforces tenant isolation at the retrieval, tool execution, state management, and API routing
133 layers. The security properties of this architecture have been formally analyzed and empirically
134 validated (Arceo & Narsing, 2026).

Kubernetes Operator

The OGX Kubernetes Operator ([OGX Contributors, 2026a](#)) provides declarative deployment through the OGXServer custom resource. A single CR specifies the distribution, replica count, storage, inference backend, and network policies. The operator manages full lifecycle reconciliation with support for both vanilla Kubernetes and OpenShift, ConfigMap-driven image overrides for fleet-wide updates, multi-architecture builds (amd64/arm64) with FIPS-compliant images, and shared instances with ABAC isolation, per-tenant namespace isolation, and hybrid topologies.

Example Usage

The following example demonstrates building a RAG agent using the standard OpenAI SDK against an OGX server. The same code works regardless of which inference provider or vector store backend is configured:

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8321/v1", api_key="unused")

# Create a vector store and upload documents
vector_store = client.vector_stores.create(name="docs")
with open("manual.pdf", "rb") as file:
    client.vector_stores.files.upload(vector_store_id=vector_store.id, file=file)

# Query with server-side RAG via the Responses API
response = client.responses.create(
    model="meta-llama/Llama-3.2-3B-Instruct",
    input="What are the installation requirements?",
    tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]}],
)
print(response.output_text)
```

Switching from a local Ollama backend to a production vLLM cluster requires changing only the server's distribution configuration—the client code above remains identical. Published tutorials demonstrate this portability across diverse enterprise backends, including IBM watsonx.ai with Milvus vector storage ([IBM Community, 2025](#)) and Oracle Cloud Infrastructure with OCI AI Blueprints ([Oracle, 2025](#)).

Research Impact Statement

OGX has realized impact through both public deployments and customer production use. Publicly documented examples under the former Llama Stack name include an intelligent OpenShift operations agent combining RAG, web search, and MCP tool integration for automated incident response ([Red Hat, 2025](#)), enterprise RAG pipelines on IBM watsonx.data with Milvus ([IBM Community, 2025](#)), and generative AI application development on Oracle Cloud Infrastructure ([Oracle, 2025](#)). Maintainers report production deployments with customers in telecommunications, semiconductor manufacturing, financial services, insurance, and consulting. The framework has been presented at Meta Connect ([Meta, 2024](#)) and IBM TechXchange ([Clyburn, 2025](#)) as a standardization layer for enterprise AI applications.

The security architecture—specifically, the multitenant isolation model combining ABAC-gated retrieval, server-side orchestration, and pluggable provider backends—was formally analyzed in a peer-reviewed publication at the ACM Conference on AI and Agentic Systems ([Arceo & Narsing, 2026](#)). OGX conforms to the Open Responses specification ([Open Responses](#)

166 [Community, 2026](#)) and serves as a reference implementation for open, vendor-neutral agentic
167 AI APIs.

168 As of June 2026, the project has over 8,400 GitHub stars, 242 contributors, 4,000 commits, and
169 68 releases across nearly two years of public development. Community engagement includes
170 weekly contributor calls, an active Discord server, and integrations contributed by external
171 organizations including Red Hat, IBM, Oracle, and Infinispan.

172 AI Usage Disclosure

173 Generative AI tools, including GitHub Copilot and Anthropic Claude models available during
174 development, were used for code completion, documentation drafting, and paper drafting.
175 Assistance was limited to generating candidate text or code that human contributors reviewed,
176 edited, tested, and validated. Core architectural decisions, API design, the security model, and
177 final paper content were made by human authors.

178 Acknowledgements

179 We thank Meta for creating and open-sourcing Llama Stack, now renamed OGX. We are grateful
180 to Red Hat for supporting the development of OGX through employee time and infrastructure
181 support. We thank the OGX contributor community for their sustained contributions to the
182 project.

183 References

- 184 Anthropic. (2026). *Model context protocol specification*. <https://modelcontextprotocol.io/specification/>
185
- 186 Arceo, F. J., & Narsing, V. P. (2026). Securing the agent: Vendor-neutral, multitenant
187 enterprise retrieval and tool use. *Proceedings of the ACM Conference on AI and Agentic*
188 *Systems*, 862–872. <https://doi.org/10.1145/3786335.3813145>
- 189 Clyburn, C. (2025). *Llama stack: Kubernetes for RAG and AI agents in generative AI*.
190 [https://mediacenter.ibm.com/media/Llama+Stack+Kubernetes+for+RAG+AI+Agents+](https://mediacenter.ibm.com/media/Llama+Stack+Kubernetes+for+RAG+AI+Agents+in+Generative+AI/1_xl78upq2)
191 [in+Generative+AI/1_xl78upq2](https://mediacenter.ibm.com/media/Llama+Stack+Kubernetes+for+RAG+AI+Agents+in+Generative+AI/1_xl78upq2)
- 192 CrewAI, Inc. (2024). *CrewAI: Framework for orchestrating role-playing autonomous AI agents*.
193 <https://github.com/crewAIInc/crewAI>
- 194 Databricks. (2025). *Mosaic AI Agent Framework*. [https://www.databricks.com/product/](https://www.databricks.com/product/machine-learning/retrieval-augmented-generation)
195 [machine-learning/retrieval-augmented-generation](https://www.databricks.com/product/machine-learning/retrieval-augmented-generation)
- 196 deepset. (2023). *Haystack: End-to-end LLM framework for building production-ready*
197 *applications*. <https://github.com/deepset-ai/haystack>
- 198 Garcia, A. (2024). *Sqlite-vec: A vector search SQLite extension*. [https://github.com/asg017/](https://github.com/asg017/sqlite-vec)
199 [sqlite-vec](https://github.com/asg017/sqlite-vec)
- 200 IBM Community. (2025). *Build RAG with Llama Stack and watsonx.data Milvus*.
201 [https://community.ibm.com/community/user/blogs/divya13/2025/05/08/build-rag-with-](https://community.ibm.com/community/user/blogs/divya13/2025/05/08/build-rag-with-llama-stack-and-watsonxdata-milvus)
202 [llama-stack-and-watsonxdata-milvus](https://community.ibm.com/community/user/blogs/divya13/2025/05/08/build-rag-with-llama-stack-and-watsonxdata-milvus)
- 203 Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang,
204 H., & Stoica, I. (2023). Efficient memory management for large language model serving
205 with PagedAttention. *Proceedings of the 29th ACM Symposium on Operating Systems*
206 *Principles*, 611–626. <https://doi.org/10.1145/3600006.3613165>

- 207 LangChain, Inc. (2023). *LangChain: Build context-aware reasoning applications*. <https://github.com/langchain-ai/langchain>
- 208
- 209 LangChain, Inc. (2024). *LangGraph: Build resilient language agents as graphs*. <https://github.com/langchain-ai/langgraph>
- 210
- 211 Llama Stack Contributors. (2025). *Llama stack*. <https://github.com/llamastack/llama-stack>
- 212 LlamaIndex. (2022). *LlamaIndex: Data framework for LLM applications*. https://github.com/run-llama/llama_index
- 213
- 214 Meta. (2024). *Llama stack: Chapter one*. <https://developers.facebook.com/m/meta-connect-developer-sessions/llama-stack-chapter-one/>
- 215
- 216 OGX Contributors. (2026a). *OGX kubernetes operator*. <https://github.com/ogx-ai/ogx-k8s-operator>
- 217
- 218 OGX Contributors. (2026b). *OGX (open GenAI stack)*. <https://github.com/ogx-ai/ogx>
- 219 Open Responses Community. (2026). *Open responses specification*. <https://www.openresponses.org/>
- 220
- 221 OpenAI. (2025). *Responses API reference*. <https://platform.openai.com/docs/api-reference/responses>
- 222
- 223 Oracle. (2025). *Accelerating enterprise gen AI applications development on OCI with Llama Stack and OCI AI blueprints*. <https://blogs.oracle.com/ai-and-datascience/accelerating-enterprise-gen-ai-applications-development-on-oci-with-llama-stack-and-oci-ai-blueprints>
- 224
- 225
- 226 Red Hat. (2025). *Generative AI applications with Llama Stack: A notebook-guided journey to an intelligent operations agent*. <https://www.redhat.com/en/blog/generative-ai-applications-llama-stack-notebook-guided-journey-intelligent-operations-agent>
- 227
- 228
- 229 Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, 41, 39–45. <https://api.semanticscholar.org/CorpusID:83459546>
- 230
- 231
- 232
- 233 Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kober, C., Shi, L., Wu, C.-S., Zhang, H., Sheng, Y., Gonzalez, J. E., Stoica, I., & Ma, W.-L. (2024). SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37.
- 234
- 235
- 236